AI ENGINEER · DATA · WEEK 5, DAY 9

Data pipelines in Microsoft Fabric

Copy only what is new, check it, label it with a model, and run it in order, in Microsoft Fabric

4 hours · instructor-led · hands-on lab

RECAP

Recap of Day 8: the agent in code

24 of 24

A. Agent in code: answers right, 3 tries a case

24 of 24

B. With a skill: skill loaded in 18 of 24

33 of 33

C. Decision flow: routes right with the fixes; 30 of 33 before

The agent is only as good as the data under it. Today: the data. 200,000 support tickets, a new day every day, and a pipeline that keeps up without anyone pressing run.

From a system that works to a system that keeps working.

AN ANALOGY

Pipeline reliability: the kitchen analogy

In the kitchen	**4 am: deliveries** check what came against the order: only today's delivery	**Bad stock aside** fish that smells wrong is put aside with a note, not served, not binned	**Prep in order** stock, then prep, then the menu: nothing starts early	**A notebook by the door** what came, what was thrown out, what it cost: read at 7
In Lab 9	**Copy job** copies only rows newer than the watermark: 600 on run 3	**tickets_quarantine** 22 rows set aside, each with its reason	**Pipeline ingest_tickets** labels start only after the checks succeed	**Run logs** Recent runs: time and status; enrich_log: model calls and cost

Same four habits, same order: Parts A, B, D and the run logs of today's lab.

Your pipeline is that kitchen: copy what is new, park what is bad, run in order, log every run.

TODAY

Today's topics: Fabric, loading, quality, enrichment, pipelines

A. Incremental loading

A copy job with a watermark

B. Data quality

Rules written once; bad rows set aside

C. LLM enrichment

200,000 tickets, 10 model calls

D. Orchestration

A pipeline, no one pressing run

E. Data agents

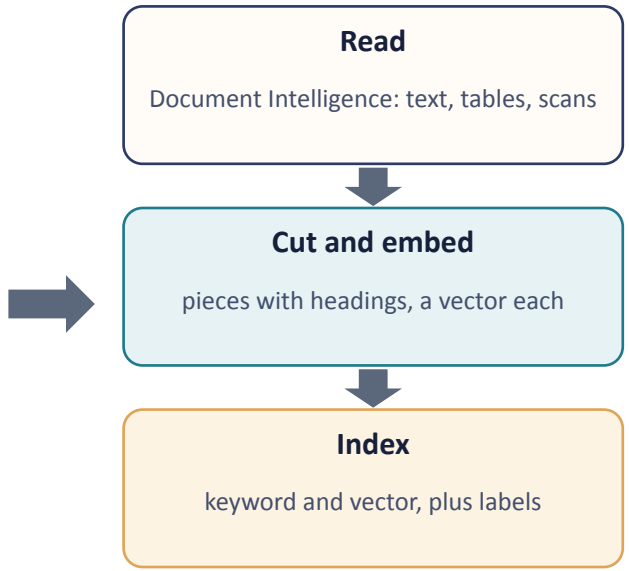
An agent over what you built

All in Microsoft Fabric, in the browser, signed in as you.

Ingestion for GenAI: documents to a trusted index

Labs 1 to 4: the 7 files

- `refund_policy_v2_2022.pdf`
- `refund_policy_v3_2024.pdf`
- `refund_claims_march_2024.xlsx`
- `support_sla_matrix.docx`
- `support_qbr_q1_2024.pptx`
- `partner_agreement_extract.pdf`
- `support_escalation_sop_scanned.pdf`



Two pieces in Lab 2's index, with its labels

`refund_policy_v3_2024.pdf`

"Refunds are paid within ..."

version 3 · 2024 · current

`refund_policy_v2_2022.pdf`

"Refunds are paid within ..."

version 2 · 2022 · not current

Azure's form made no labels, so there the 2022 policy ranks next to the 2024 one.

Three traps we hit, none raised an error; the same steps for every source

Count what went in

Minimal extraction: 6 pieces, the scan read as nothing. Standard: 8.

Keep it fresh

A deleted file stayed in the index after a re-run.

Labels

Azure's form dropped version and current-or-not. Blob, SharePoint, OneLake: the same steps.

Labs 1 to 4 built each step by hand; Azure's forms do the same steps, and the same traps remain.

Labs 1 to 4, now in Azure: the same steps, Azure's services

D1. Read	**Document Intelligence**	was pypdf and docx2txt
D2. Index	**Azure AI Search**	was Chroma, Qdrant, FAISS
D3. Search better	**keyword, meaning, reranker**	built in; was BM25 and FlashRank
D4. Answer	**a Foundry model, as Lab 8**	was Llama on Databricks
Same data	**7 files, Lab 3 and 4 questions**	so the scores compare

THE POINT

Data ingestion for GenAI: read, cut, label, index. The capstone runs on these services.

What changed, in numbers: Labs 1 to 4 against Azure

Scanned PDF read	**825 characters**	Lab 1: 0, and no error
Word table	**column names kept**	Lab 1: lost
Search, meaning only	**0.851 nDCG@10, 20 questions**	Lab 3: 0.75, 18 questions
Search, all stages	**0.928 nDCG@10**	Lab 3: 0.92
Right facts	**20 of 20, every run**	Lab 4: 17 or 18

ASK THE ROOM

The start is higher and the climb is smaller. Where did the gain come from: reading, or searching?

What still breaks in Azure: the old traps

The old policy

The 2022 policy is second under the 2024 one until a filter takes it out

The reranker

Better on average, worse on 3 of 20 questions: R02 fell from 0.88 to 0.30

The free tier

Read 2 of the PowerPoint's 3 slides, with no warning

Cited, still wrong

The Singapore question answered in 2 of 4 runs; once with real citations

Refusal wording

"Not mentioned" in its own words is not the refusal sentence code looks for

Code checks that every source is real; a person still reads the near misses.

Labs 1 to 4

Labs 1 to 4 in Azure: your documents

Pick by time · in the browser, in teams · labs1-4_azure folder · handbook Labs1-4_in_Azure.pdf

Build

- D1. Read the scan and the Word table: Document Intelligence
- D2. Your container, knowledge base and source
- D3. Count and search your index
- D4. An agent answers, cites, refuses
- D5. Add a new file and re-index

Pick by time

- D1: 15 min, D2: 15
- D3: 15, D4: 15
- D5: 10
- Optional, the same in code: 60
- Lab 9, the Fabric pipeline: 90

1

Microsoft Fabric

Where today's pipeline lives

Microsoft Fabric: the main workloads on one platform

Data Factory
copy jobs, pipelines

Analytics
lakehouse, notebooks

Databases
SQL inside Fabric

Real-Time Intelligence
events and streams

IQ
data agent: Part E, class F2

Power BI
reports

Copilot, in every workload: writes notebook code, SQL and pipelines from a sentence

OneLake

One store under every workload. Nothing is copied between them.

`tickets_clean`

- ← the dataflow writes it
- → SQL reads it
- → Power BI reads it
- → a data agent reads it

Governance, across every workload: permissions, sensitivity labels, auditing, the OneLake catalog

used in Lab 9

IQ's data agent runs on the class's paid F2: the trial refuses it

Lab 9 uses three of them: Data Factory, Analytics, and IQ's data agent in Part E.

Microsoft Fabric: workspace, lakehouse, notebooks, pipelines

Workspace: FreshCart Lab 9 - your name

Lakehouse	`freshcart_lake`	files + tables
Copy job	`(Part A)`	brings data in
Dataflow Gen2	`quality_tickets`	checks the rows
Notebook	`lab9_2_enrich`	Python on Spark: the labels
Pipeline	`ingest_tickets`	runs them in order

Inside freshcart_lake

Files anything, as it arrived

Tables

tickets_raw	tickets_clean
tickets_quarantine	ticket_labels
tickets_enriched	enrich_log

A table one item writes, the next item reads: the copy job fills tickets_raw, the dataflow reads it.

Capacity

The compute you rent. Your workspace runs on a 60-day trial; Part E's data agent runs on the class's paid F2.

Signed in as you

The copy job reads the database as you; the notebook calls the model as you. No keys.

Databricks had the same ideas in Weeks 1 to 4: lakehouse, notebooks, jobs.

OneLake and capacities: where data lives, what pays

OneLake: one lake for the whole organisation

FreshCart Lab 9 - your name

trial capacity, Central India

`lakehouse freshcart_lake`

Files	any format
Tables	Delta Parquet
SQL endpoint	read-only T-SQL

FreshCart data agent demo (trainer)

F2 capacity, West US 3

`lakehouse: a copy of Lab 9`

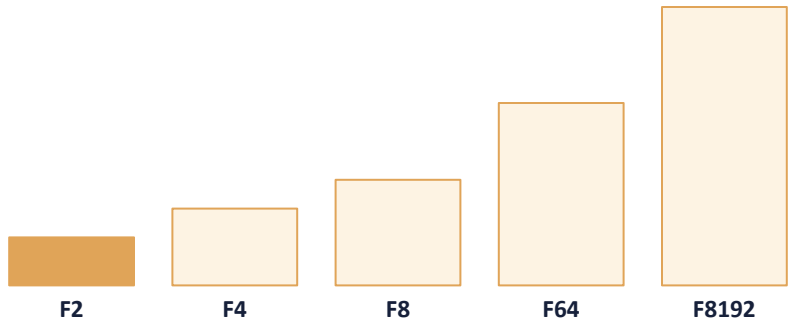
Files	any format
Tables	Delta Parquet
SQL endpoint	read-only T-SQL

Amazon S3, ADLS

or another workspace

Shortcut: a link to data somewhere else, read in place, not copied.

Capacity: the compute you pay for



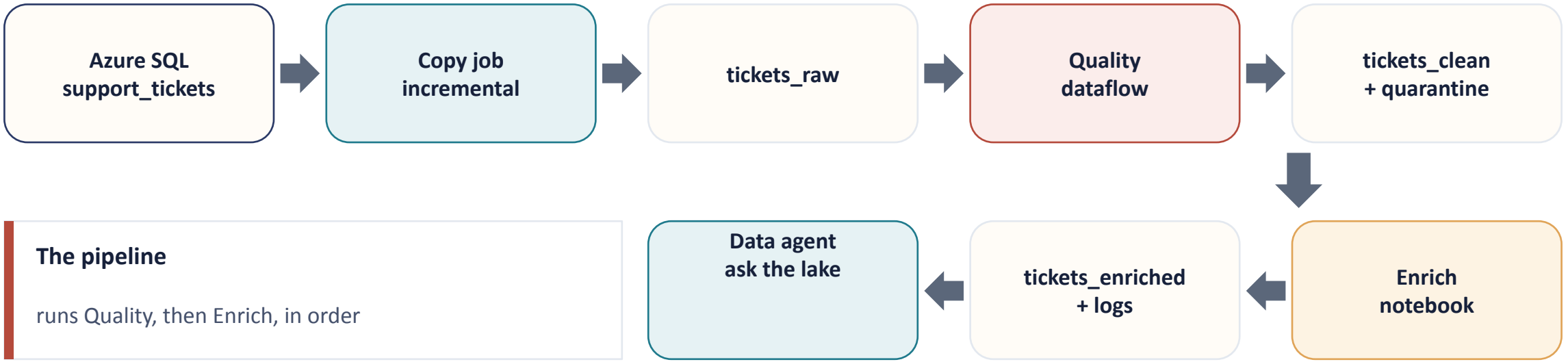
Billed per second while running, can be paused. One region each.

Trial

Free for 60 days, but some features refuse it: our data agent failed on the trial, ran on an F2.

Bronze, silver, gold: Lab 9's tickets_raw, tickets_clean and tickets_enriched are the same three layers.

Lab architecture: from database to answers



The pipeline

runs Quality, then Enrich, in order

200,000 tickets to start; then a new day of 600 arrives, with problems planted in it by the trainer.

2

Incremental loading

A copy job with a watermark

Incremental loading: three methods

The source table, the morning of run 3

ticket_id	created_at	what happened since run 2
TKT-0000042	2024-04-23	refund edited: 103.17 to 999.00
TKT-0091377	2024-08-14	deleted at the source
TKT-0150210	2024-11-02	nothing
TKT-0200431	2024-12-31	new: one of the 600 new tickets

red line: the watermark, created_at 2024-12-30 23:59. Run 3 reads below it only.

	Full reload	Watermark	Change data capture
	everything, every run	rows below the line	the database's change log
TKT-0000042	copied	missed	copied
TKT-0091377	removed	missed	removed
TKT-0150210	copied again	left alone	left alone
TKT-0200431	copied	copied	copied
	200,600 rows	**600 rows**	**changes only**
	on run 3: everything again	on run 3	must be switched on

Merge key ticket_id: a ticket copied twice updates its row, never duplicates it. Idempotent: running twice gives the same result, so retries are safe and cheap.

Our copy job uses a watermark on created_at: fast, and blind to edits and deletes.

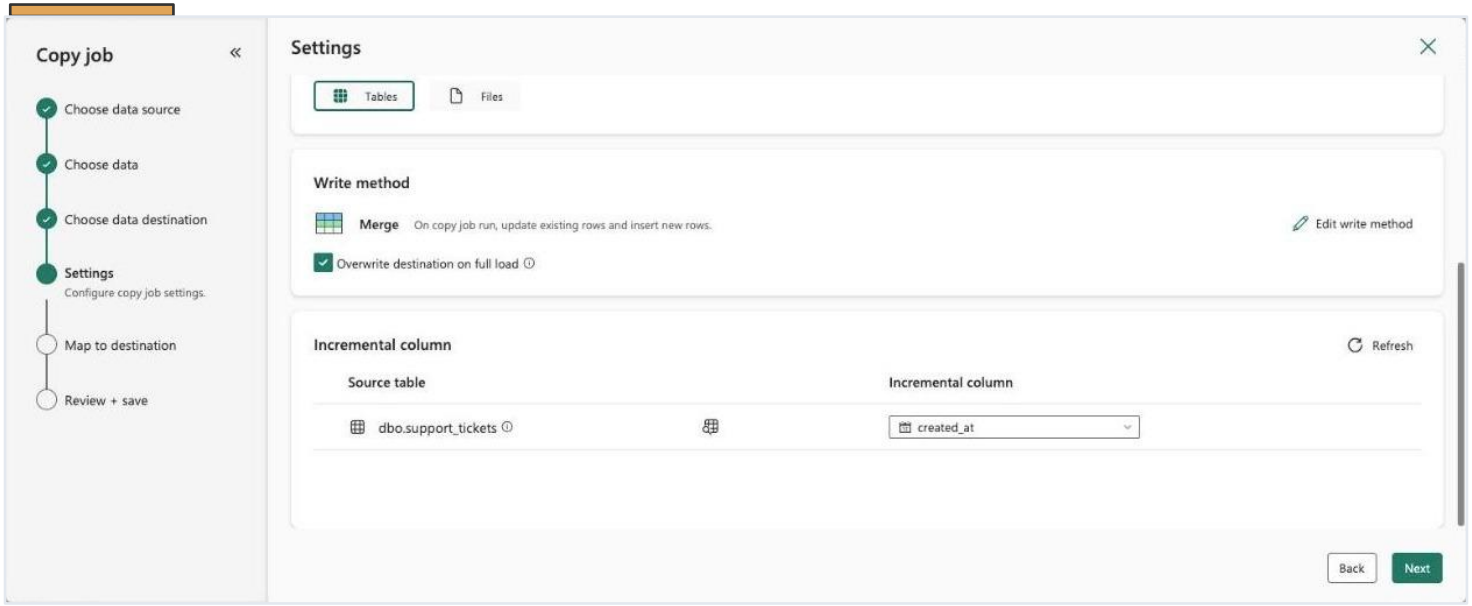
Full vs incremental load: our three runs



Run 1: full load, watermark 2024-12-30 23:59. Run 2: nothing new, nothing copied. Run 3: the trainer added 31 December; only those 600 came.

The watermark is the newest created_at already copied. Next run starts there.

Copy job settings: incremental column and merge key



Incremental column: created_at

Copy rows newer than the last run's newest.

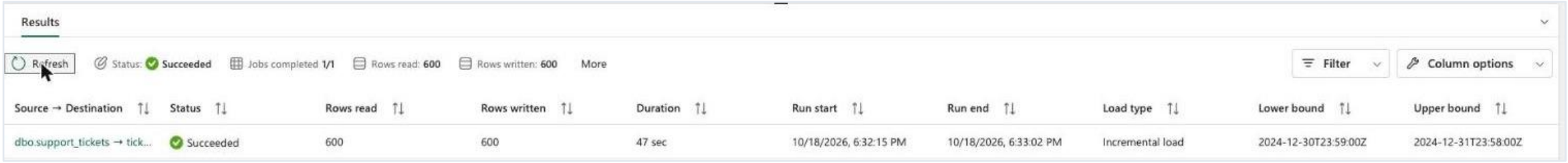
Merge on ticket_id

A ticket copied twice updates; it is never duplicated.

Signed in as you

Organizational account: no password.

Copy job run history: what was copied



Rows read 600, rows written 600. Lower bound 2024-12-30 23:59, upper bound 2024-12-31 23:58: exactly the new day.

Read the run's own record before you trust the table.

Watermark limits: edits and deletes

999.00

TKT-0000042 refund at the source, after the edit

103.17

the same ticket in our lakehouse

Why

It was created in April. The watermark only looks after 30 December.

An updated_at column

Copy on last change, not on creation.

Change data capture

The database records every insert, edit and delete.

A periodic full compare

Weekly: count and checksum source against lake.

Edits and deletions are the changes everyone forgets, and the answers keep flowing.

3

Data quality

Rules written once

Data quality: expectations and quarantine

Rows arrive from tickets_raw

- **TKT-0000187** customer C-20419, refund 0.00
- **TKT-0000512** customer missing
- **TKT-0003390** refund -15.00
- **TKT-0007714** category 'groceries'

200,600 rows in run 1

A failing row is parked as

no customer	9
negative refund	4
unknown category	6
resolved before first response	3

red: rows each rule stopped in run 1

tickets_clean

200,578

passed all four, go on to the model

tickets_quarantine

22

kept with the reason, for example
TKT-0000512: no customer

A bad row is parked with its reason, never silently dropped: someone can fix it and send it back.

Dataflow Gen2: cleaning data by clicking

quality_tickets: applied steps

- **Get data**
  tickets_raw, from freshcart_lake
- **Remove duplicates**
  `on ticket_id`
- **Conditional column**
  `rule_customer`
- **Conditional column**
  `rule_refund`
- **Conditional column**
  `rule_order`
- **Conditional column**
  `rule_category`
- **Custom column**
  `failed_rules: the reasons joined`
- **Reference + filter**
  `tickets_clean: failed_rules blank`
- **Reference + filter**
  `tickets_quarantine: not blank`

What the steps make, row by row

ticket_id	rule_customer	rule_refund	failed_rules
TKT-0000187	null	null	(blank)
TKT-0000512	no customer	null	no customer
TKT-0003390	null	negative refund	negative refund

`tickets_clean`

200,578 lakehouse, Replace

`tickets_quarantine`

22 lakehouse, Replace

When to use it

Row-by-row cleaning an analyst can read and change later.

When not

Very large tables, or logic that needs code and tests: use a Spark notebook.

Run 1 took 97 s, run 2 took 67 s, same totals: bad rows parked with a reason, never dropped.

Lineage in Fabric and Purview: where a number came from

How the Lab 9 data moves. Fabric's lineage view draws items: freshcart_lake is one box holding the tables.



Where did 200,578 come from? Walk left.

Change tickets_clean: what breaks? Walk right.

In the table

Delta history: every write, with its time and rows.

In the run

Run logs and Recent runs: what each run read, wrote and cost.

Sensitivity labels

Pass from a semantic model to its reports; stay on exported files.

Endorsement

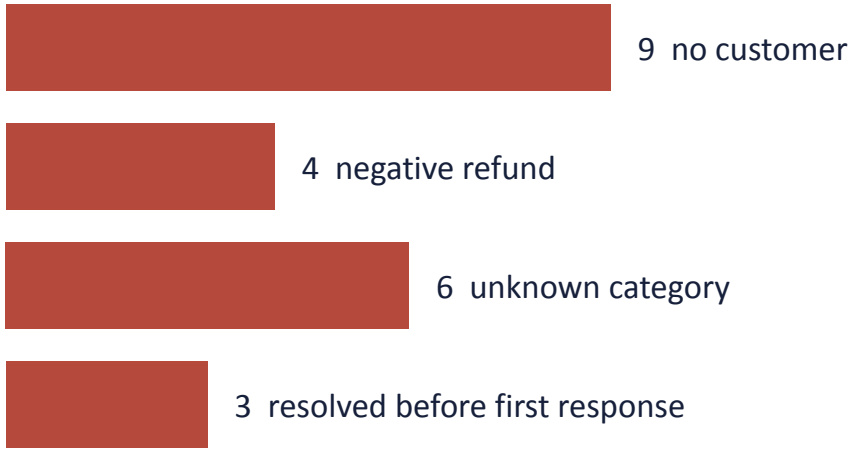
Promoted, certified or master data, with an owner: which table to trust.

Purview: the same lineage and a catalog, across the whole company, not one workspace.

Lineage answers two questions: where did this number come from, and what breaks if I change it?

Data quality rules: four expectations

```
RULES = {   # key = problem, value = what passes
  "no customer": "customer_id IS NOT NULL",
  "negative refund": "refund_amount >= 0",
  "unknown category": "category IN (...8...)",
  "resolved before first response":
      "resolution_minutes >= first_response_minutes"}
```



22 planted problems in the new day

A rule that cannot be judged (a missing value) counts as failed. The reasons are kept as text, so SQL and agents can read them.

Bad rows are parked with a reason, never dropped silently.

Idempotent loads: running it twice

200,600

run 1: new tickets checked

200,578

clean, and 22 parked

0

run 2: new tickets; totals unchanged

```
new = raw.join(handled, "ticket_id", "left_anti")      # only tickets we have not seen
DeltaTable...merge(new, "t.ticket_id = s.ticket_id")   # a ticket arriving twice is stored once
```

Idempotent: running it again is always safe. That is what makes retries cheap.

Lineage and run logs

run_at	new_rows	clean_total	quarantine_total	seconds
run 1	200,600	200,578	22	53.2
run 2	0	200,578	22	39.9

run_log

One row per run: what came, what stayed, what was parked, how long it took.

DESCRIBE HISTORY

Delta keeps every change to a table, with rows inserted and updated: lineage for free.

Tomorrow's Day 10 alarms read these logs.

QUICK CHECK

Before the break, three questions

1. Run 2 copied 0 rows. What must be true about the copy for that to be safe?

2. A ticket created in April is edited today. Trace what our pipeline does with it.

3. Why park bad rows in a table instead of dropping them?

Take answers in chat. Do not move on until at least two people have answered.

Break

10 minutes. After the break: a model over 200,000 rows, and a pipeline that runs itself.

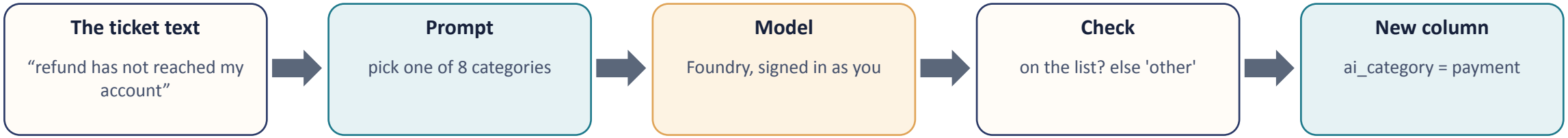
4

LLM enrichment at scale

Without paying 200,000 times

LLM enrichment: a model as a pipeline step

One row through the step



More labels from our run

“item missing from the bag”	missing_item
“driver could not find the address”	late_delivery
“food was cold”	other
“charged twice for one order”	payment

What it costs: rows x tokens per row x price

$4.97

one call for each of 200,578 rows, about 22 hours

$0.000248

10 calls: the rows held only 10 distinct texts

What goes wrong: 429s, answers off the list, a new answer on a rerun, the provider changing the model. Try 1,000 rows first.

Cut the cost: cache each text · smaller model · batches · retry, then fail loudly

Treat every model call as a paid API call that can fail: count it, cache it, retry it.

Storing model output: validate, cache, record provenance

One row of tickets_enriched

ticket_id	text	ai_category	ai_model	ai_prompt_version	ai_labelled_at
TKT-0000187	refund has not reached my account	payment	gpt-4.1-mini	v1	2026-10-18 ...

constrained: one of 8 categories

provenance: explain this label months later

Validate

The model answers outside the list, say “billing issue”: it is stored as other, flagged off_list in ticket_labels. Malformed output is routine.

Cache by content

```
key = model|prompt version|text
```

Same text, same key: no second call, same label every time. Change the prompt to v2 and every text is labelled again.

Log the cost: enrich_log

run	calls	cost
1	10	$0.000248
2	0	$0, all cache

Run 1: 559 tokens in, 15 out. Tomorrow's cost alarm reads this table.

Caching also turns a non-deterministic step into a repeatable one.

Cost per row: caching by distinct text

10 calls

run 1: $0.000248, labelled every ticket

0 calls

run 2: all from the cache, $0

$4.97

one call per ticket: 200,578 calls

At about 0.4 seconds a call, one call per ticket is about 22 hours one after another. The cache key is model + prompt version + text: same input, same label, no second call.

Estimate on a sample and multiply before you run the whole table.

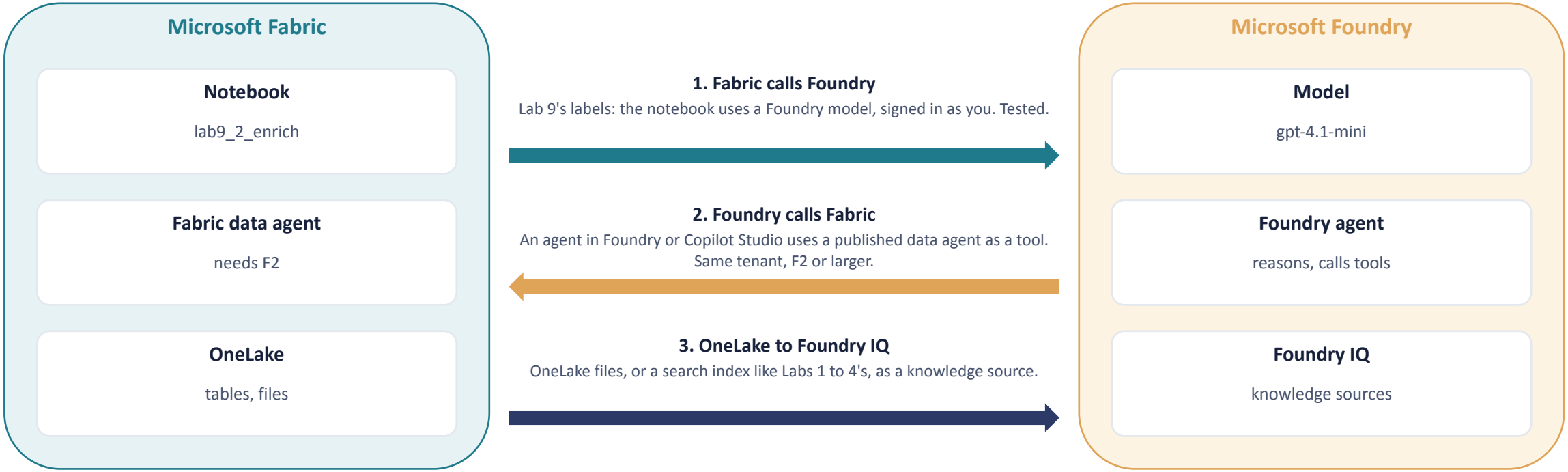
Enrichment results: what the labels found

item missing from the bag	missing_item	packaging was torn on arrival	other
spillage across the whole order	spillage	app would not let me reorder	account
driver could not find the address	late_delivery	wrong size delivered	wrong_item
food was cold	other	refund has not reached my account	payment
charged twice for one order	payment	order arrived late	late_delivery

Example: of 20,117 clean tickets saying 'item missing from the bag', support recorded 2,492 as missing_item. 1 in 8 is what chance gives: the recorded categories were never reliable.

The label did not just add a column; it showed a data problem nobody had seen.

Foundry with Fabric: three ways to join them



Copilot Studio or your own app, then Teams and Copilot: where people ask

The capstone joins them: data in Fabric, agent in Foundry, people in Teams.

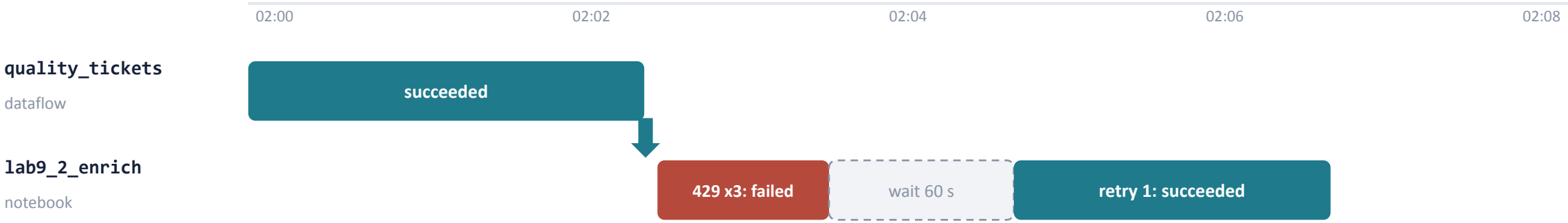
5

Orchestration

A pipeline

Orchestration: order, retries, safe re-runs, records

One night of ingest_tickets, drawn as a timeline (example)



Order

Step 2 never starts before step 1 finished: the arrow on success.

Retries

A failed step is tried again, not the whole night: retry 1, after 60 s.

Safe to repeat

Running twice gives the same tables: watermark, anti-join, merge, cache.

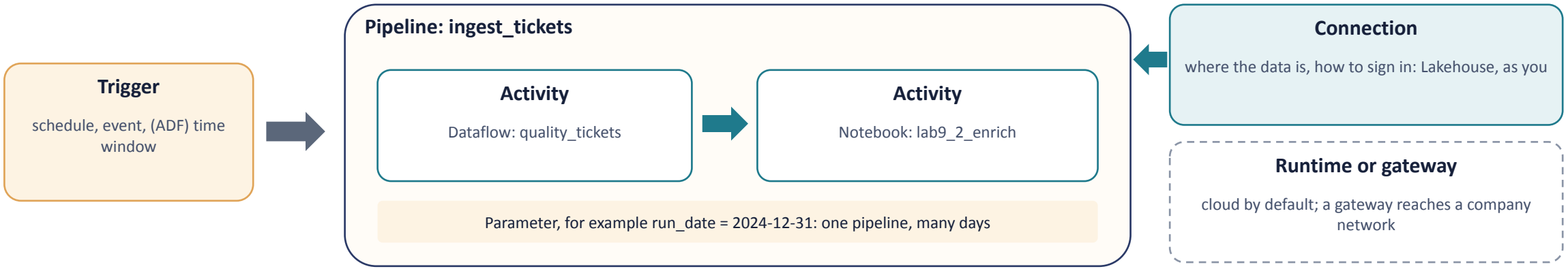
A record

What ran, what failed, what it cost: run_log, enrich_log, the run history.

A timer calling a script at 02:00 would run step 2 even after step 1 failed.

If your pipeline is a timer calling a script, you have scheduling, not orchestration.

Data Factory building blocks: ADF and Fabric



Monitoring: every run, every activity

activity	name	status
Dataflow	`quality_tickets`	Succeeded

Our run with only the dataflow step: 128 s for the whole pipeline.

ADF calls a connection a linked service and adds datasets; Fabric folds both into the connection.

Azure Data Factory vs Fabric pipelines: the next generation

Azure Data Factory

- a separate tool with its own screen
- you manage the compute that moves the data
- runs SSIS packages (SQL Server's data tool)
- changes go live only when you click Publish

paid per run and per hour of data moved

Data Factory in Fabric

- inside your workspace, next to your lakehouse
- copy job and Dataflow Gen2 built in
- can send a Teams or Outlook message as a step
- no machines or Publish; a gateway reaches company servers

paid by the capacity, while it is on

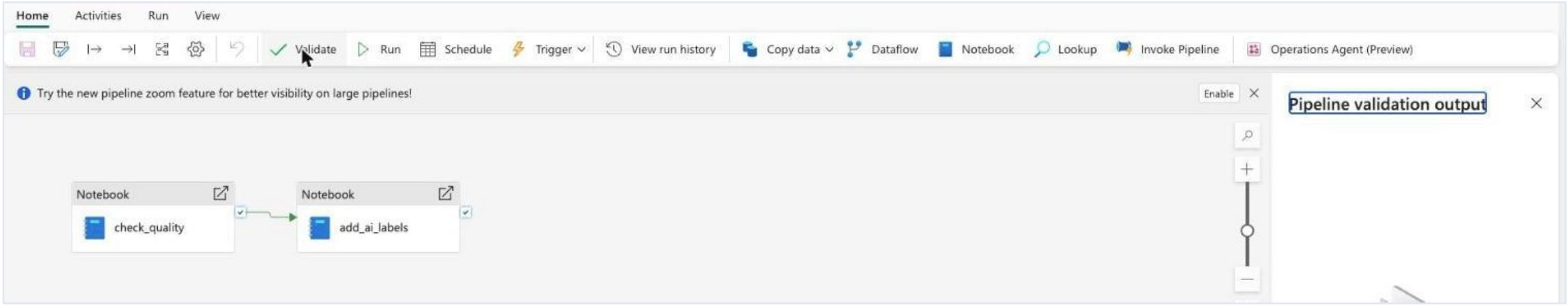
Same idea, new name

pipeline	→	pipeline	linked service	→	connection	change capture	→	copy job	mapping data flow	→	Dataflow Gen2

about 90% of ADF's activities exist in Fabric

Lab 9's copy job and pipeline are Fabric's Data Factory. Build new work in Fabric; keep ADF where it already runs.

Fabric pipelines: steps in order



Order

Labels start only after the checks succeed (the green arrow).

Retry

Each step retries once after 60 seconds: model calls fail now and then.

128 seconds

The dataflow step alone, in our test; the notebook adds its own time.

The copy job keeps its own schedule: inside a pipeline it needs a second sign-in.

Fabric IQ: one shared meaning for your data

Tables in OneLake

- `customers`
- `orders`
- `tickets_clean`

Ontology: the business, described once

Customer	*places* →	**Order**
Order	*has* →	**Ticket**

a rule: a late delivery is a Ticket whose category is late_delivery

Who reads it

- **Fabric data agent**
- **Foundry agent, Fabric IQ tool**

What it is

A Fabric workload, in preview. Its ontology names the things your business deals with (customer, order, ticket), their fields and links, and ties each one to tables in OneLake.

How agents use it

A question in business words becomes a query, so the agent does not need to guess from column names. A data agent can add an ontology as a source; a Foundry agent can call it as a tool.

When to use it

Many tables and many agents that must use the same words. Not needed for Lab 9: one lakehouse, clear column names and one paragraph of instructions.

Describe the business once, and every agent reads the same meaning.

Fabric data agents: when to prefer them over a Foundry agent

Fabric data agent

- no code: pick tables, write instructions
- writes SQL, DAX or KQL itself, and only reads
- runs as the person asking, with their permissions
- publish to Copilot, Teams or Copilot Studio

needs a paid F2 or larger capacity; the free trial refuses it

Your own agent in Foundry

- your choice of model
- any tools: a policy file, an email, other systems
- can act, not only read
- your own checks and tests, as in Lab 10

built in the portal or in code; needs no Fabric capacity of its own

Pick by the question

“How many tickets were parked, and why?”	**data agent, Lab 9 Part E: 22 tickets; no customer 9, unknown category 6, negative refund 4, other 3**
“Should order 1850 get a refund?”	**Foundry agent: reads the policy and past claims, then decides (Lab 7)**

Both: a Foundry agent can call a published data agent as one of its tools (preview).

Ask about Fabric data with a data agent; build in Foundry when the job needs other tools or actions.

Work IQ: what an agent can know about your work

A person's Microsoft 365

- Mail and calendar
- Files and SharePoint
- Teams chats
- People

Work IQ

answers in plain language,
always as the signed-in person,
seeing only what they can see

Who asks it

- **Microsoft 365 Copilot**
- **Copilot Studio agents**
- **Foundry agents (preview)**
- **your own app**

What it is

Microsoft's layer over a person's work: mail, meetings, files, chats and people, plus Dynamics 365 and Power Platform data (preview). An agent asks in plain words and gets an answer.

Safe by design

Every request runs as the signed-in person and honours their Microsoft 365 permissions. Admins set rules on what agents may read or change.

Cost and setup

Billed by use, so the API needs no Microsoft 365 Copilot licence. An admin approves it once for the whole organization.

"What did the delivery partner promise in last week's meeting?" goes to Work IQ. "How many late-delivery tickets yesterday?" goes to the data agent.

Work IQ knows your work, the data agent knows your tables, Foundry IQ knows your documents.

Fabric data agents: how a question becomes a query

A student asks

"How many tickets were put in quarantine, and for which reasons?"

The agent reads

its instructions (rules, terms, example questions) and the six table descriptions

```
SELECT failed_rules, COUNT(*)
FROM tickets_quarantine
GROUP BY failed_rules
```

reason	rows
no customer	9
negative refund	4
unknown category	6
resolved before first response	3

Read the query under each answer; ask more than once.

22

Right: 9, 4, 6 and 3. It ran the query as the student, with the student's own rights.

44

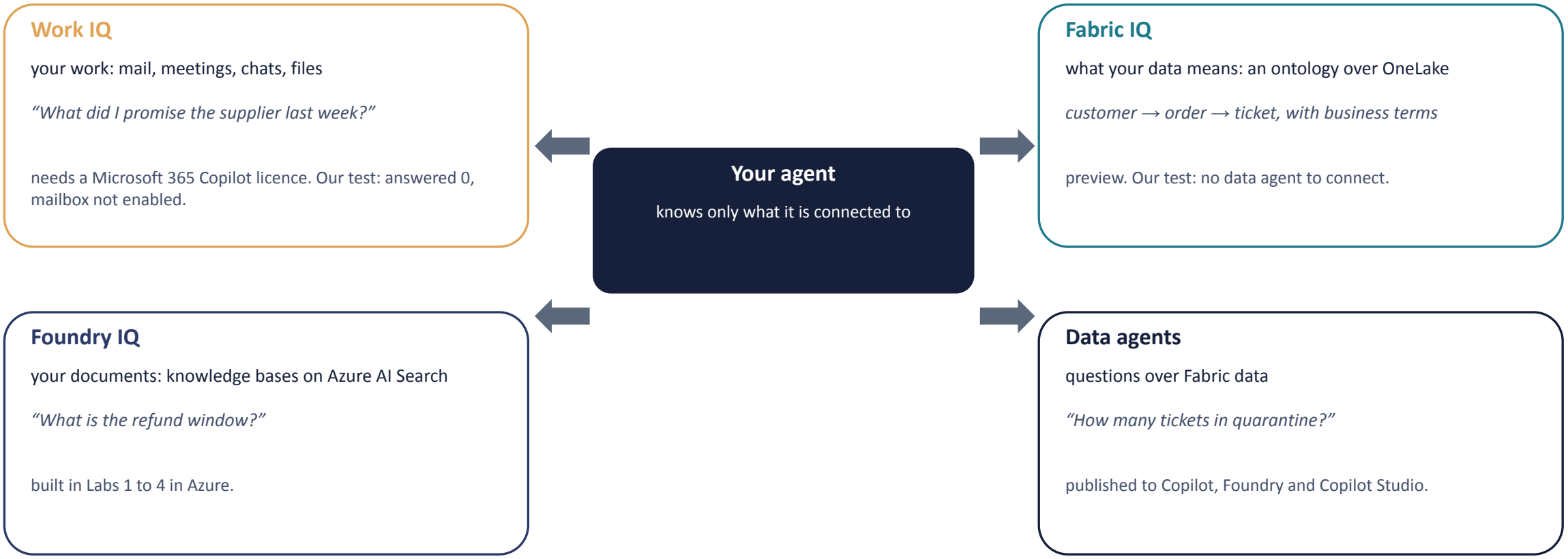
Our student agent once said 44: its SQL added the overall total to the four parts. It got 4 of 5 right.

Publish: to Microsoft 365 Copilot, Copilot Studio, Foundry agents

Capacity: a paid F2 or larger, tenant settings on; the trial refuses it

An agent over data is only as good as the descriptions of that data.

Work IQ, Fabric IQ, Foundry IQ: what an agent can know



An agent knows only what it is connected to: each IQ is one connection you choose.

Data agents: asking questions of the lakehouse

37 of 45

answers right: 5 questions, 9 tries each

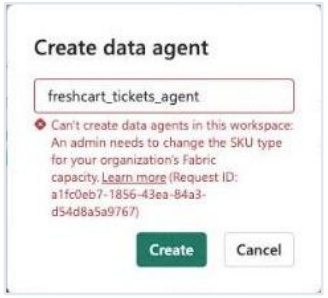
45 of 45

with one sentence describing enrich_log

The miss

Asked for model calls, it counted 200,578 labelled rows.

```
def query_lake(sql: str) -> dict:
    """Run ONE read-only SELECT on the lakehouse ... Tables: tickets_clean (...), enrich_log (...)"""
```



Fabric's own Data agent does this without code, and each team builds one in Lab 9, on the class's paid capacity: the trial refuses it. Same lesson either way: an agent over data is only as good as the descriptions of that data.

QUICK CHECK

Three questions before the lab

1. Your model step fails after 900 of 1,000 texts. With our cache, what does the retry cost?

2. Someone edits the prompt. What happens to the cache, and to the cost of the next run?

3. Why does the pipeline retry each step once, not three times?

Take answers in chat, then into the lab.

Lab 9

A pipeline that runs without you, in Microsoft Fabric

90 minutes · your Fabric workspace · lab9_fabric folder · handbook Fabric_Lab9_pipeline.pdf

Build

- Step 0: a lakehouse, freshcart_lake
- A. Copy job: incremental on created_at, merge on ticket_id
- B. Dataflow quality_tickets: four rules, keep, park
- C. lab9_2_enrich: labels with a cache and a cost
- D. Pipeline: the dataflow, then the labels notebook
- E. A data agent per team, in the class workspace

Prove it works

- Run the copy twice: 0 rows the second time
- After the new day: only 600 rows
- Quality twice: same tables; labels twice: 0 calls
- Find TKT-0000042 and explain it
- Data agent: 5 questions, read the SQL

Lab 9 results

A. Copy job	**200,000 / 0 / 600**	rows in runs 1, 2 and 3
B. Quality	**200,578 clean, 22 parked**	same totals on run 2
C. Labels	**10 calls, $0.000248**	then 0 calls; one per ticket would be $4.97
D. Pipeline	**128 s**	the dataflow step alone, in our test
E. Data agent	**4 of 5, one run each**	Q1 counted 22 twice: read the SQL

ASK THE ROOM How would you catch the edit to TKT-0000042? What would it cost to run every night?

TAKE AWAY

What carries into tomorrow

Copy what is new

A watermark and a merge key; edits and deletions need their own plan.

Check, keep, park

Rules written once; bad rows set aside with a reason.

A model at scale

Constrain, validate, cache by content, record provenance and cost.

Run in order

Safe to repeat, retried, and logged every time.

Tomorrow: it runs every night, and nobody yet knows whether last night made it worse.